Chapitre 4 : Fonctions, Méthodes de Srings, de Listes, de Tuples et de Sets

Python Functions

- En Python, une fonction est un groupe d'instructions liées qui accomplissent une tâche spécifique.
- Les fonctions aident à diviser notre programme en parties plus petites et modulaires. À mesure que notre programme devient de plus en plus grand, les fonctions le rendent plus organisé et plus facile à gérer.
- Elles évitent la répétition et rendent le code réutilisable.

- Syntaxe d'une fonction

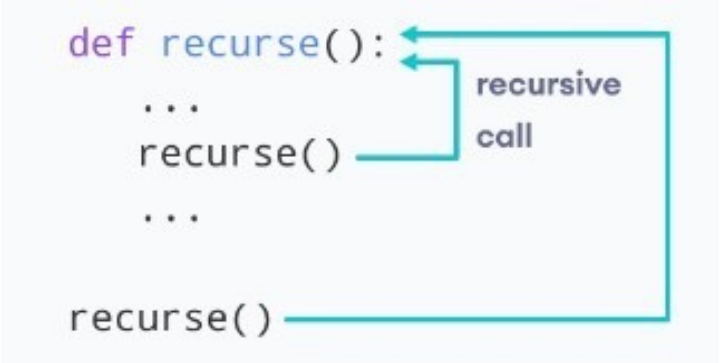
```python
def function_name(parameters):
  """docstring"""
  statement1
  statement2
  statement3
  return value
```

```python
def abs_val(num):
    """This function returns the absolute
    value of the entered number"""

    if num >= 0:
        return num
    else:
        return -num

var=-9

print("|",var,"| = ",abs_val(var))
```

```
| -9 | =  9
```

Récursivité

- La récursivité est le processus qui consiste à définir une fonction en termes d'elle-même (c'est-à-dire une fonction qui s'appelle elle-même).
- Elle est généralement utilisée pour résoudre un problème complexe en le divisant en versions simplifiées de lui-même.
- L'image suivante illustre le fonctionnement d'une fonction récursive appelée « recurse ».




```python
def factorial(num):
  if(num==0):
    return 1
  else:
    return num*factorial(num-1)

var=5

print(var,"! = ",factorial(var))
```

```
5 ! =  120
```

Anonymous / Lambda Function (1/1)

- En Python, une fonction anonyme est une fonction définie sans nom. Les fonctions anonymes sont créées avec le mot-clé lambda, d'où leur nom : Lambda function.

- Syntaxe d'une fonction Lambda en Python :

lambda arguments: expression

- On utilise les fonctions lambda lorsqu'une fonction sans nom est nécessaire temporairement (par exemple, lorsqu'on passe une fonction en argument à une autre fonction).

Anonymous/Lambda Function (1/2)

Exemple : La fonction filter() en Python prend une fonction et une liste en arguments et renvoie une liste contenant les éléments pour lesquels la fonction lambda retourne True.

```python
# Program to filter out only the even items from a list
my_list = [1, 5, 4, 6, 8, 11, 3, 12]

new_list = list(filter(lambda x: (x%2 == 0) , my_list))

print(new_list)
```

```
[4, 6, 8, 12]
```

Ici, la fonction filter() applique la fonction :
lambda x: x % 2 == 0 à chaque élément de la liste my_list et ne garde que ceux pour lesquels la condition est vraie (les nombres pairs dans cet exemple).

Les listes (1/2)

- Les listes sont similaires à des tableaux de taille dynamique, comme les vecteurs en C++ et les ArrayList en Java.
- Les éléments d'une liste sont ordonnés, modifiables et permettent des valeurs en double.
- En Python, une liste est créée en plaçant les éléments à l'intérieur de crochets [], séparés par des virgules.

```
my_list = [1, 2, 3, 4]
```

- On peut utiliser l'opérateur d'indexation [] pour accéder à un élément dans une liste, l'indexation commençant à 0.
- Python permet également l'indexation négative pour ses séquences. L'index -1 fait référence au dernier élément.

Les listes (1/2)

append()	**Ajoute un élément à la fin de la liste**
extend()	**Ajoute tous les éléments d'une liste à une autre liste.**
insert()	**Insère un élément à un index défini dans la liste.**
remove()	**Supprime le premier élément trouvé avec la valeur spécifiée.**
pop()	**Retourne et supprime un élément à l'index donné.**
clear()	**Supprime tous les éléments de la liste.**
index()	**Retourne l'index du premier élément correspondant à la valeur spécifiée.**
count()	**Retourne le nombre d'occurrences d'un élément dans la liste.**
sort()	**Trie les éléments de la liste dans l'ordre croissant.**
reverse()	**Inverse l'ordre des éléments dans la liste.**
copy()	**Retourne une copie superficielle de la liste.**

Les Tuples (1/3)

- **Un tuple en Python est similaire à une liste. La différence entre les deux réside dans le fait qu'on ne peut pas modifier les éléments d'un tuple une fois qu'il a été attribué.**

- **Un tuple est créé en plaçant tous les éléments à l'intérieur de parenthèses () :**

my_tuple = (1, 2, 3)

- **Pour accéder à un élément d'un tuple, on utilise l'opérateur d'indexation [], où l'index commence à 0 :** *my_tuple[2]*.

Avantages des tuples par rapport aux listes :

1.Performance : Parcourir un tuple est plus rapide qu'une liste.

2.Protection contre les modifications : Si vous avez des données qui ne changent pas, les implémenter sous forme de tuple garantit qu'elles restent protégées contre les modifications.

Tuple (2/3)

Contrairement aux listes, les tuples en Python ont un nombre très limité de méthodes disponibles. Cela est dû à leur nature immuable, qui restreint les opérations pouvant être effectuées. Les deux principales méthodes pour les tuples sont :

- **count() : Renvoie le nombre de fois qu'une valeur spécifique apparaît dans un tuple.**
- **index() : Recherche une valeur spécifique dans le tuple et renvoie la position où elle a été trouvée.**

PS : Les tuples étant immuables (non modifiables), on ne peut pas :

•Ajouter ou supprimer des éléments (pas de méthodes comme append() ou remove()).

•Modifier leur contenu.

Tuples (3/3)

Cependant, les tuples peuvent être utilisés avec des fonctions natives Python comme :

- **len() : Renvoie le nombre d'éléments dans le tuple.**
- **sorted() : Renvoie une liste triée des éléments du tuple (le tuple lui-même reste inchangé).**
- **min() et max() : Renvoient respectivement la plus petite et la plus grande valeur.**
- **sum() : Calcule la somme des éléments, si ce sont des nombres.**
- **in : Vérifie si un élément est présent dans le tuple.**

Set (1/3)

Un ensemble (set) est une collection non ordonnée d'éléments. Chaque élément d'un ensemble est unique (pas de doublons) et doit être immuable (ne peut pas être modifié).

Cependant, un ensemble lui-même est mutable : on peut y ajouter ou supprimer des éléments.

Un ensemble est créé en plaçant tous les éléments à l'intérieur d'accolades {}, séparés par des virgules, ou en utilisant la fonction intégrée set().

```
my_set = {1, 2, 3}
my_set = set([1, 2, 3, 2])
```

Pour ajouter des éléments à un ensemble, on utilise la fonction add() : my_set.add(7).
Pour supprimer un élément d'un ensemble, on utilise les fonctions discard() ou remove() :

```
my_set.discard(2)
my_set.remove(6)
```

Set (2/3)

add()	Ajoute un élément à l'ensemble.
clear()	Supprime tous les éléments de l'ensemble.
copy()	Retourne une copie de l'ensemble.
difference(other_set)	Retourne un nouvel ensemble contenant les éléments présents dans le premier ensemble mais pas dans le second (équivalent à -).
Difference_update(other_set)	Supprime de l'ensemble actuel les éléments communs avec un autre ensemble. Modifie directement l'ensemble.
discard()	Supprime un élément spécifique sans générer d'erreur s'il n'existe pas.
intersection(other_set)	Retourne un nouvel ensemble contenant les éléments communs aux deux ensembles (équivalent à &).
Intersection_update(other_set)	Modifie l'ensemble actuel pour ne conserver que les éléments communs avec un autre ensemble.
isdisjoint(other_set)	Vérifie si les deux ensembles n'ont aucun élément en commun.

Set (3/3)

issubset(other_set)	Vérifie si tous les éléments de l'ensemble sont présents dans un autre ensemble.
issuperset(other_set)	Vérifie si l'ensemble contient tous les éléments d'un autre ensemble.
pop()	Supprime et retourne un élément aléatoire de l'ensemble (car les ensembles sont non ordonnés).
remove()	Supprime un élément spécifique. Génère une erreur si l'élément n'existe pas.
Symmetric_difference(other_set)	Retourne un nouvel ensemble contenant les éléments non communs aux deux ensembles (équivalent à ^).
Symmetric_difference_update(other_set)	Modifie l'ensemble actuel pour contenir les éléments qui sont dans l'un ou l'autre ensemble, mais pas dans les deux.
union(other_set)	Retourne un nouvel ensemble contenant les éléments des deux ensembles
update(other_set)	Ajoute plusieurs éléments provenant d'un itérable (liste, tuple, etc.) à l'ensemble actuel.

Dictionary (1/2):

Un dictionnaire en Python est une collection ordonnée d'éléments (à partir de Python 3.7). Chaque élément d'un dictionnaire est constitué d'une paire clé/valeur.
Créer un dictionnaire est aussi simple que de placer des éléments à l'intérieur d'accolades {}, séparés par des virgules :

My_dict= {'nom': Ahmed', 'âge': 26}

Un dictionnaire utilise les clés pour accéder à ses valeurs. Les clés peuvent être utilisées soit à l'intérieur de crochets [], soit avec la méthode get().

Les dictionnaires sont modifiable : on peut ajouter de nouveaux éléments ou modifier la valeur des éléments existants en utilisant un opérateur d'affectation.

Ajouter un élément : my_dict['address'] = 'Centre Ville'.

Mettre à jour une valeur : my_dict['age'] = 27.

On peut supprimer un élément particulier dans un dictionnaire en utilisant la méthode pop()

Dictionary (2/2):

clear()	Supprime tous les éléments du dictionnaire.
copy()	Retourne une copie superficielle du dictionnaire.
fromkeys(iterable, value)	Crée un nouveau dictionnaire avec des clés provenant d'un itérable et une valeur identique pour toutes les clés.
get(key, default)	Retourne la valeur associée à une clé. Si la clé n'existe pas, retourne une valeur par défaut.
items()	Retourne une liste contenant des paires clé/valeur sous forme de tuples.
keys()	Retourne une liste contenant toutes les clés du dictionnaire.
pop(key, default)	Supprime une clé et retourne sa valeur. Si la clé n'existe pas, retourne une valeur par défaut si fournie.
popitem()	Supprime et retourne la dernière paire clé/valeur du dictionnaire. (V3.7)
setdefault(key, default)	Retourne la valeur associée à une clé. Si la clé n'existe pas, l'ajoute avec une valeur par défaut.
update(other_dict)	Met à jour le dictionnaire avec les paires clé/valeur d'un autre dictionnaire ou d'un itérable.
values()	Retourne une liste contenant toutes les valeurs du dictionnaire.

Strings (1/5):

Les chaînes de caractères (strings) en Python sont entourées soit de guillemets simples, soit de guillemets doubles.
On peut créer des chaînes en entourant des caractères avec des guillemets simples ou doubles :

```
my_string = 'Python'
```

Les triples guillemets peuvent être utilisés en Python pour représenter des chaînes multi-lignes et des docstrings.
On peut accéder à des caractères individuels en utilisant l'indexation str[i] :

```
my_string[0]        #output : P
```

Les chaînes sont immuables, ce qui signifie que les éléments d'une chaîne ne peuvent pas être modifiés une fois qu'ils ont été attribués.
La concaténation désigne l'opération de joindre deux ou plusieurs chaînes en une seule. Cela se fait avec l'opérateur +

Strings (2/5):

capitalize()	Retourne une copie de la chaîne avec le premier caractère en majuscule et tous les autres en minuscules.
casefold()	Retourne une copie de la chaîne en minuscules
center()	Retourne une copie de la chaîne centrée
count()	Retourne le nombre d'occurrences d'une sous-chaîne dans la chaîne
endswith()	Retourne True si la chaîne se termine par une sous-chaîne donnée
expandtabs()	Remplace les tabulations dans la chaîne par un nombre spécifié d'espaces (par défaut, 8 espaces).
find()	Retourne l'index de la première occurrence d'une sous-chaîne, ou -1 si elle n'est pas trouvée.
format()	Formate la chaîne en remplaçant les champs {} par les valeurs données.
format_map()	Formate la chaîne en utilisant un mapping (par exemple un dictionnaire) pour remplacer les champs {}.
index()	Retourne l'index de la première occurrence d'une sous-chaîne, ou lève une exception si elle n'est pas trouvée.

Strings (3/5):

• isalnum()	• Retourne **True** si tous les caractères de la chaîne sont alphanumériques.
• isalpha()	• Retourne **True** si tous les caractères de la chaîne appartiennent à l'alphabet.
• isascii()	• Retourne **True** si tous les caractères de la chaîne sont des caractères ASCII.
• isdecimal()	• Retourne **True** si tous les caractères de la chaîne sont des décimaux.
• isdigit()	• Retourne **True** si tous les caractères de la chaîne sont des chiffres.
• isidentifier()	• Retourne **True** si la chaîne est un identifiant valide.
• islower()	• Retourne **True** si tous les caractères de la chaîne sont en minuscules.
• isnumeric()	• Retourne **True** si tous les caractères de la chaîne sont numériques.
• isprintable()	• Retourne **True** si tous les caractères de la chaîne sont imprimables.
• isspace()	• Retourne **True** si tous les caractères de la chaîne sont des espaces blancs.
• istitle()	• Retourne **True** si la chaîne suit les règles d'un titre (première lettre de chaque mot en majuscule).
• isupper()	• Retourne **True** si tous les caractères de la chaîne sont en majuscules.

Strings (4/5):

join()	Convertit les éléments d'un itérable en une chaîne de caractères.
ljust()	Retourne une version de la chaîne alignée à gauche.
lower()	Convertit une chaîne en minuscules.
lstrip()	Retourne une version de la chaîne avec les espaces à gauche supprimés.
maketrans()	Retourne une table de traduction à utiliser pour les traductions.
partition()	Retourne un tuple où la chaîne est divisée en trois parties.
replace()	Retourne une chaîne où une valeur spécifiée est remplacée par une autre valeur spécifiée.
rfind()	Recherche une valeur spécifiée dans la chaîne et retourne la dernière position où elle a été trouvée.
rindex()	Retourne une version de la chaîne alignée à droite.
rjust()	Retourne un tuple où la chaîne est divisée en trois parties.

Strings (5/5):

rsplit()	Divise la chaîne en utilisant le séparateur spécifié et retourne une liste.
rstrip()	Retourne une version de la chaîne avec les espaces à droite supprimés.
split()	Divise la chaîne en utilisant le séparateur spécifié et retourne une liste.
splitlines()	Divise la chaîne aux sauts de ligne et retourne une liste.
stratswith()	Retourne True si la chaîne commence par la valeur spécifiée.
strip()	Retourne une version de la chaîne avec les espaces au début et à la fin supprimés.
swapcase()	Inverse les cas : les minuscules deviennent majuscules et vice versa.
title()	Convertit le premier caractère de chaque mot en majuscule.
translate()	Retourne une chaîne traduite selon une table de traduction.
upper()	Convertit une chaîne en majuscules.
zfill()	Remplit la chaîne avec un nombre spécifié de zéros (0) au début.

Slicing (1/3) : Utilisation de base des tranches

L'opération de découpage (slicing) est utilisée pour accéder à une plage d'éléments dans tout type de données séquentielles en utilisant l'opérateur « : »

Exemple : Sélectionner une sous-liste à partir de la liste nums :

```
nums = [10, 20, 30, 40, 50, 60, 70, 80, 90]
some_nums = nums[2:7:1]
print(some_nums)
```

```
[30, 40, 50, 60, 70]
```

La syntaxe est « start : stop : step »

- **Start** fait référence à l'indice de l'élément qui sert de point de départ de notre découpage.
- **Stop** fait référence à l'indice de l'élément où il faut s'arrêter, mais sans inclure cet élément dans le découpage.
- **Step** permet de sélectionner chaque n-ième élément dans la plage définie par start

Slicing (2/3) : Exemples

Prendre les **n** premiers éléments d'une liste.

```
nums = [10, 20, 30, 40, 50, 60, 70, 80, 90]
some_nums = nums[:5]
print(some_nums)
```

```
[10, 20, 30, 40, 50]
```

Prendre tout les derniers éléments à partir du **n^{ième} élement**

```
nums = [10, 20, 30, 40, 50, 60, 70, 80, 90]
some_nums = nums[3:]
print(some_nums)
```

```
[40, 50, 60, 70, 80, 90]
```

Prendre tous les éléments d'une liste sauf les **n** derniers.

```
nums = [10, 20, 30, 40, 50, 60, 70, 80, 90]
some_nums = nums[:-2]
print(some_nums)
```

```
[10, 20, 30, 40, 50, 60, 70]
```

Slicing (3/3): Negative Step & Reversed List

Le **pas négatif** (-1) est utilisé pour inverser l'ordre des éléments d'une liste en Python.

```python
nums = [10, 20, 30, 40, 50, 60, 70, 80, 90]
some_nums = nums[::-1]
print(some_nums)
```

```
[90, 80, 70, 60, 50, 40, 30, 20, 10]
```

Cela va du dernier élément au premier élément.
En raison de cette particularité, les indices de début (**start**) et de fin (**stop**) doivent également être spécifiés de droite à gauche.
Exemple :

```python
nums = [10, 20, 30, 40, 50, 60, 70, 80, 90]
some_nums = nums[-2::-1]
print(some_nums)
```

```
[80, 70, 60, 50, 40, 30, 20, 10]
```